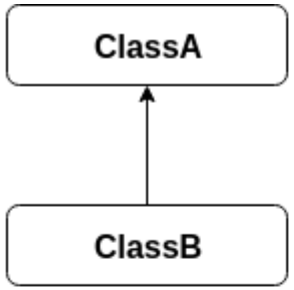
Chapter - 8

INHERITANCE

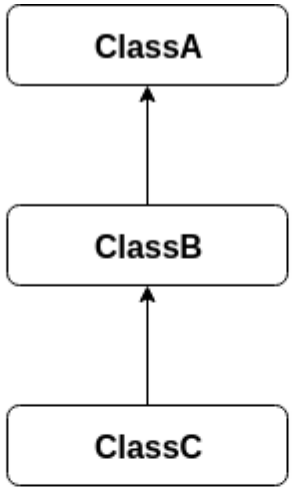
What is Inheritance?

- The mechanism of deriving a new class from an old one is called Inheritance.
- The old class is called **'BASE'** class, and the new one is called **‘DERIVED’** class.
- **Advantages of Inheritance**
  - Code Reusability
  - Improved Code Maintenance
  - Method Overriding / Polymorphism
  - Extensibility / Scalability

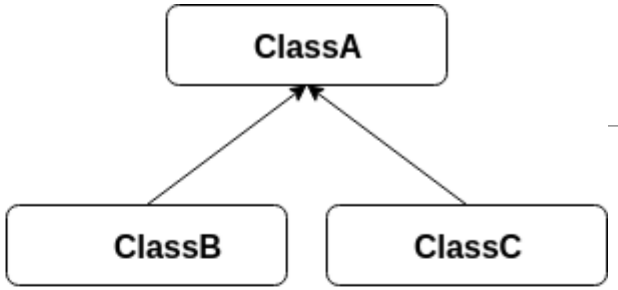
Types



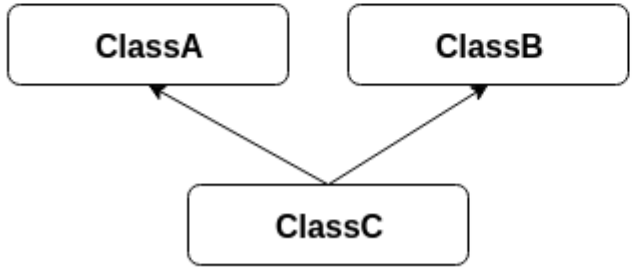
Single Inheritance



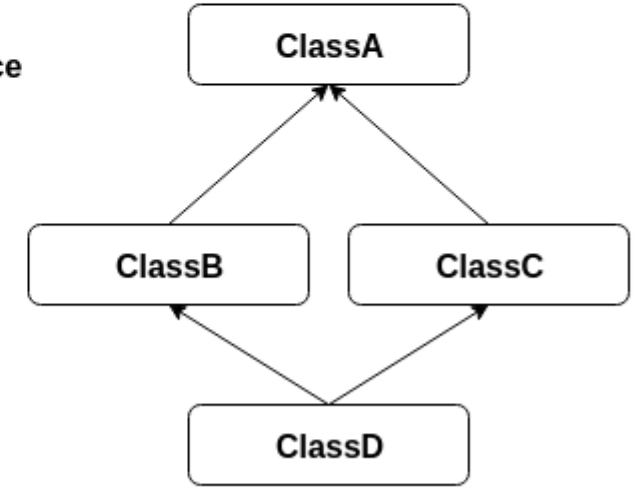
Multilevel Inheritance



Hierarchical Inheritance



Multiple Inheritance



Hybrid Inheritance

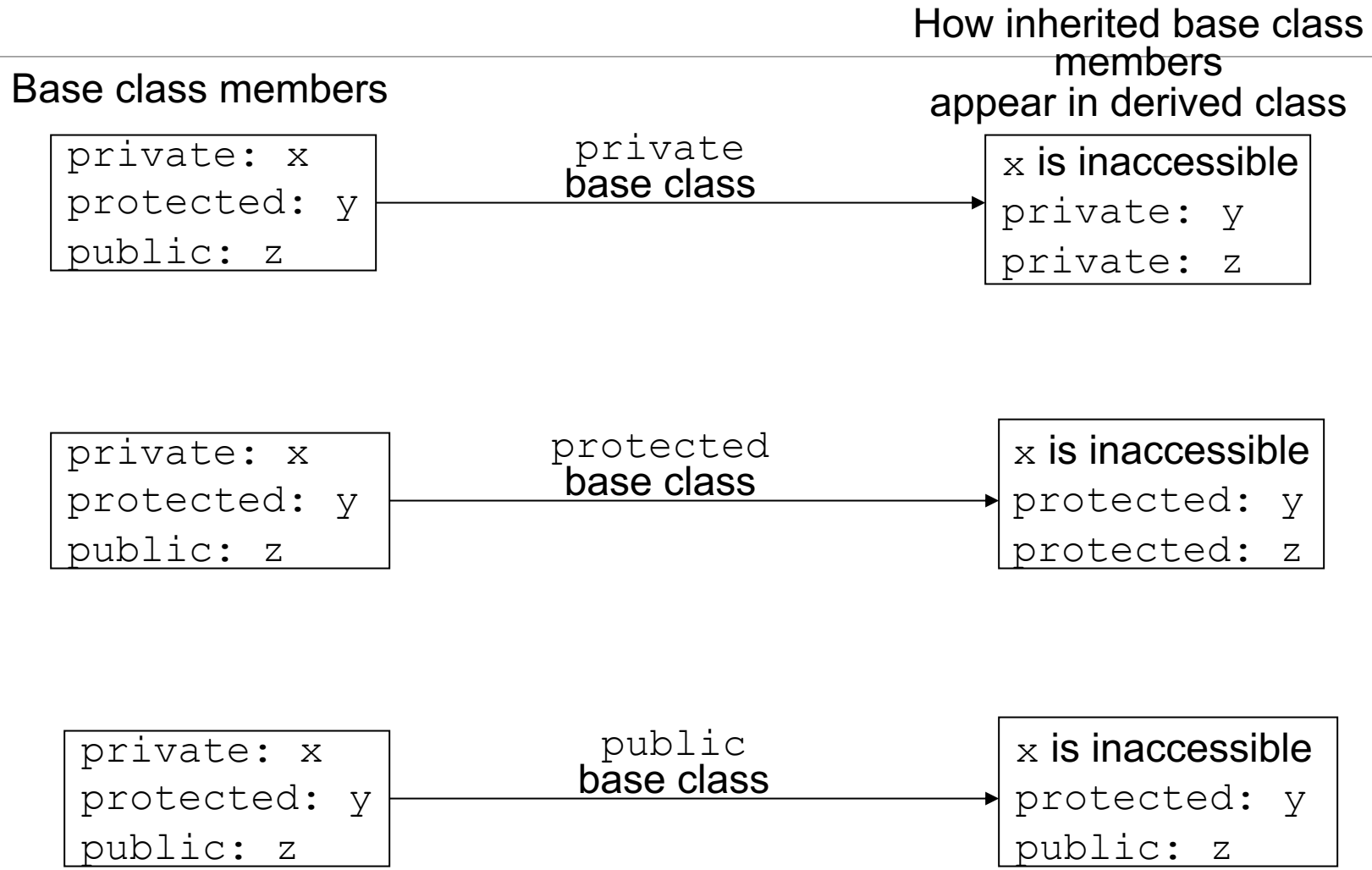
Inheritance – Terminology and Notation

```
class derived-class-name  : visibility-mode base-class-name
{
        .....//
        .....//   members of derived class
        .....//
};
```

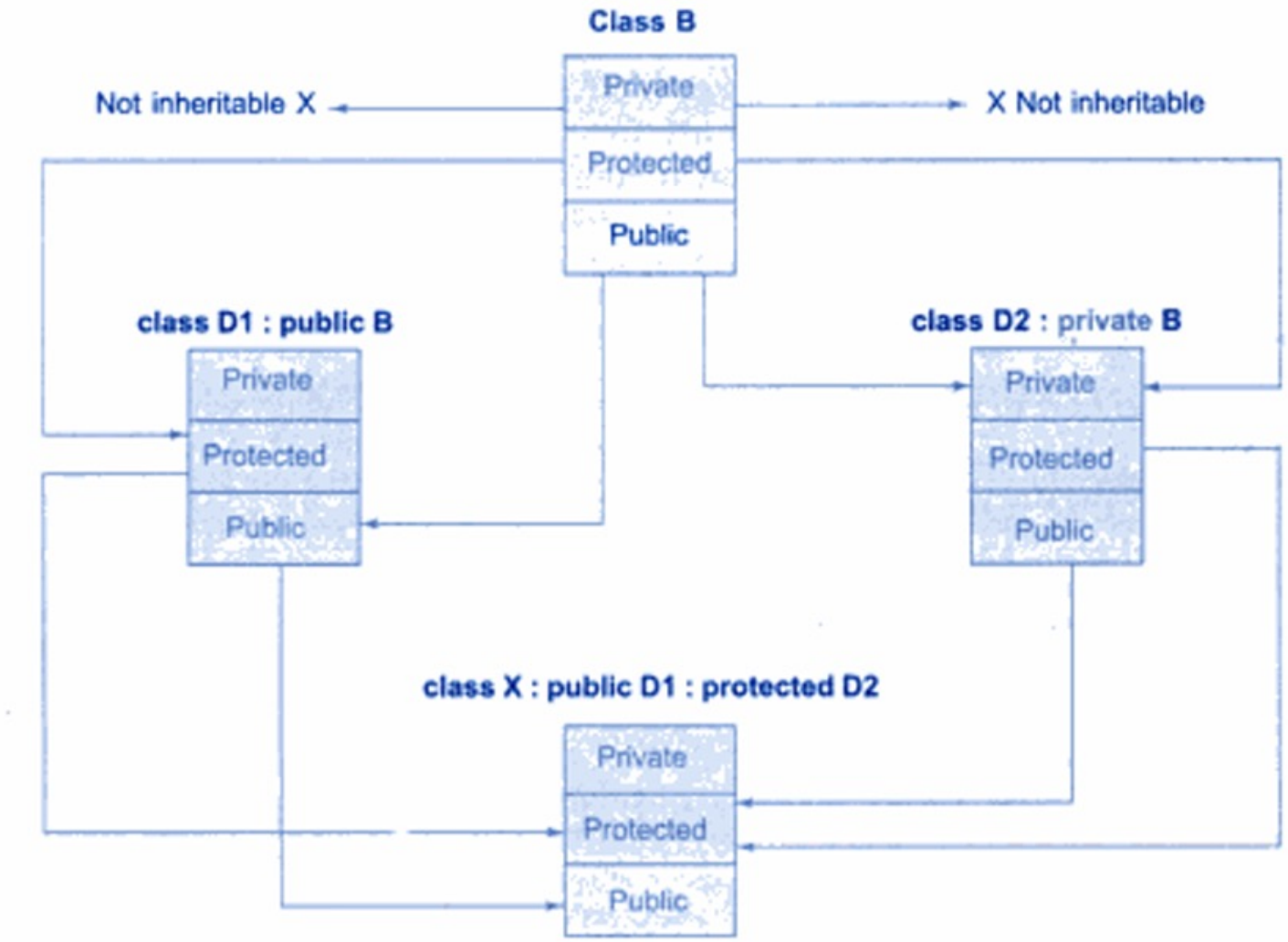
Examples:

```
class ABC: private XYZ        // private derivation
{
        members of ABC
};

class ABC: public XYZ         // public derivation
{
        members of ABC
};

class ABC: XYZ                // private derivation by default
{
        members of ABC
};
```

Inheritance vs. Access



Inheritance vs. Access



Single Inheritance

```
#include <iostream>
using namespace std;
// Base class
class Animal {
public:
    void eat() {
        cout << "I can eat!" << endl;
    }
};
// Derived class (single inheritance)
class Dog : public Animal {
public:
    void bark() {
        cout << "I can bark! Woof woof!" << endl;
    }
};
```

```
int main() {
    Dog myDog;
    myDog.eat();   // Inherited from Animal
    myDog.bark();  // Own method
    return 0;
}
```

Making a private member inheritable

Use protected instead of private

```
class Base {
private:
    int privateValue;   //  Not inheritable

protected:
    int protectedValue; //  Inheritable (accessible in Derived)

public:
    // Setter for private value
    void setPrivateValue(int p) {
        privateValue = p;
    }

    // Setter for protected value
    void setProtectedValue(int q) {
        protectedValue = q;
    }

    void showPrivate() {
        cout << "Private Value (only accessible in Base): " <<
privateValue << endl;
    }
};
```

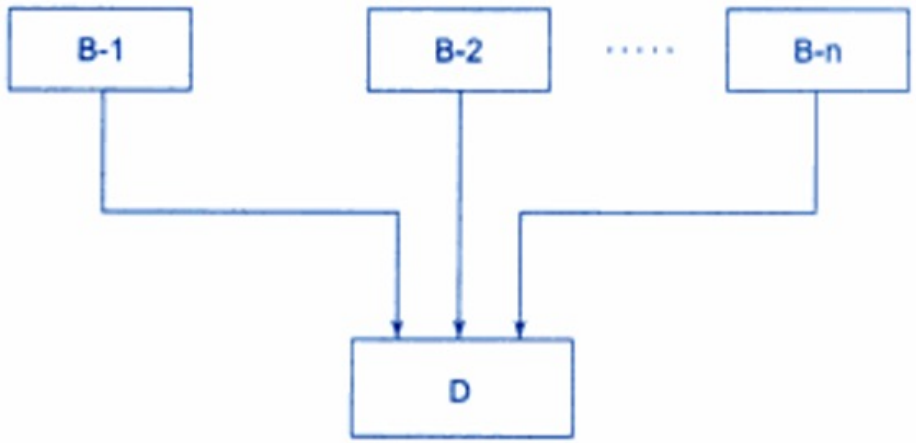
```
class Derived : public Base {
public:
    void showValues() {
        // cout << privateValue;   //  ERROR: private not accessible
        cout << "Protected Value (accessible in Derived): " <<
protectedValue << endl;
    } };
int main() {
    Derived d;
    // Set values through setter methods
    d.setPrivateValue(10);
    d.setProtectedValue(20);
    d.showPrivate();   //  Access private through Base's public method
    d.showValues();    //  Direct access to protected member
    return 0; }
```

Multilevel inheritance

```
class Student {
private:
    int rollNo;
public:
    void getRollNo(int r) {
        rollNo = r;
    }
    void putRollNo() {
        cout << "Roll Number: " << rollNo << endl;
    }
};
// First derived class
class Marks : public Student {
protected:
    int mark1, mark2;
public:
    void getMarks(int m1, int m2) {
        mark1 = m1;
        mark2 = m2;
    }
    void putMarks() {
        cout << "Marks in Subject 1: " << mark1 << endl;
        cout << "Marks in Subject 2: " << mark2 << endl;
    } };
```

```
class Result : public Marks {
public:
    void displayResult() {
        int total = mark1 + mark2;
        float percentage = total / 2.0;
        cout << "Total Marks: " << total << endl;
        cout << "Percentage: " << percentage << "%";
    }
};
// Driver code
int main() {
    Result r;

    r.getRollNo(101);   // from Student
    r.getMarks(85, 90); // from Marks

    cout << "--- Student Details ---" << endl;
    r.putRollNo();
    r.putMarks();
    r.displayResult();  // from Result

    return 0;
}
```

Multiple Inheritance



The syntax of a derived class with multiple base classes is as follows:

```
class D: visibility B-1, visibility B-2 ...
{
      .....
      .....(Body of D)
      .....
};
```

Multiple Inheritance

```
// First base class
class MathSkills {
public:
    void add(int a, int b) {
        cout << "Sum: " << a + b << endl;
    }
};

// Second base class
class ScienceSkills {
public:
    void experiment() {
        cout << "Performing experiment..." << endl;
    }
};
```

```
class SmartStudent : public MathSkills, public ScienceSkills {
public:
    void study() {
        cout << "Studying..." << endl;
    }
};
int main() {
    SmartStudent student;
    // Using methods from both parent classes
    student.add(5, 3);       // From MathSkills
    student.experiment();    // From ScienceSkills
    student.study();         // Own method
    return 0;
}
```

Ambiguity Resolution:

```
// Two base classes with same method name
class Father {
public:
    void show() {
        cout << "Father's method" << endl;
    } };
class Mother {
public:
    void show() {
        cout << "Mother's method" << endl;
    } };
// Derived class from both
class Child : public Father, public Mother {
public:
    void display() {
        cout << "Child's method" << endl;     }};
```

```
int main() {
    Child c;
    // These would cause ambiguity error:
    // c.show(); // Error: ambiguous call
    // Resolving ambiguity:
    c.Father::show();  // Call Father's method
    c.Mother::show();  // Call Mother's method
    c.display();       // Call Child's own method
    c.showBoth();      // Call method that
resolves ambiguity
    return 0; }
```

Hierarchical Inheritance

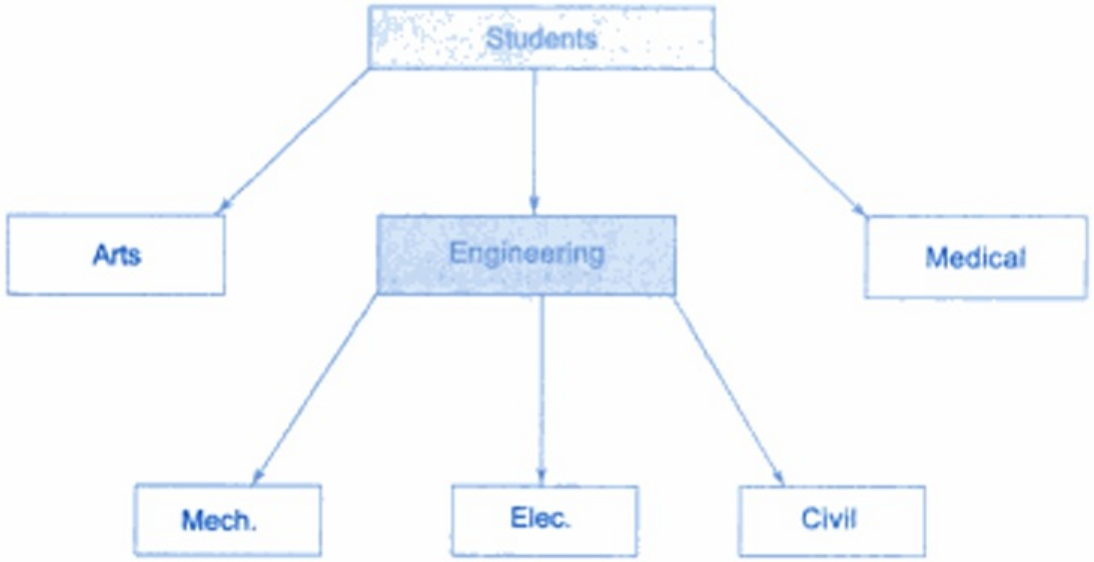


Fig. 8.9 ⇔ *Hierarchical classification of students*

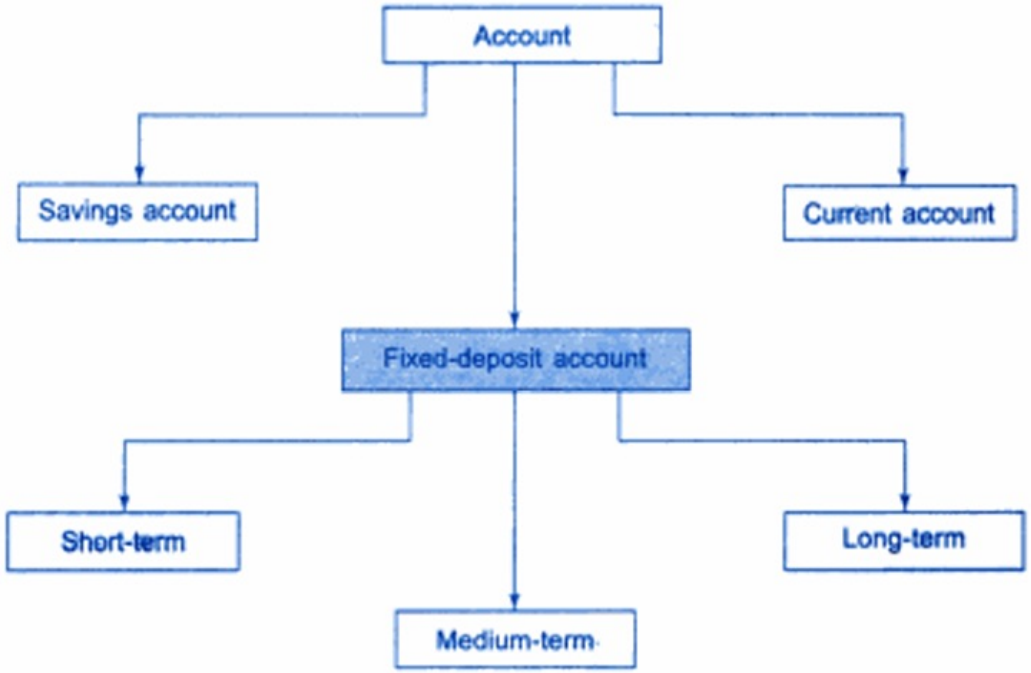


Fig. 8.10 ⇔ *Classification of bank accounts*

Hierarchical Inheritance

```
class Student {
public:
    void getRoll(int r) {
        roll = r;                }
    void showRoll() {
        cout << "Roll: " << roll << endl;     }
private:
    int roll; };
class Science : public Student {
public:
    void getMarks(int s1, int s2) {
        sub1 = s1;
        sub2 = s2;                          }
    void showResult() {
        cout << "Science Marks: " << sub1 << ", " << sub2 << endl;
        cout << "Total: " << sub1 + sub2 << endl;     }
private:     int sub1, sub2; };
```

```
class Arts : public Student {
public:
    void getMarks(int s1, int s2) {
        sub1 = s1;
        sub2 = s2;                                  }
    void showResult() {
        cout << "Arts Marks: " << sub1 << ", " << sub2 << endl;
        cout << "Total: " << sub1 + sub2 << endl;     }
private:  int sub1, sub2; };
int main() {
    Science sci;
    Arts art;
     cout << "Science Student:" << endl;
    sci.getRoll(101);
    sci.getMarks(85, 90);
    sci.showRoll();
    sci.showResult();
    cout << "\nArts Student:" << endl;
    art.getRoll(102);
    art.getMarks(78, 82);
    art.showRoll();
    art.showResult();
        return 0; }
```

Hybrid inheritance

```
class Person {
protected:
    string name;
    int age;
public:
    void setPerson(string n, int a) {
        name = n;
        age = a;     }
    void showPerson() {
        cout << "Name: " << name << ", Age: " << age << endl;
    } };
// Derived from Person
class Student : public Person {
protected:
    int rollNo;
public:
    void setStudent(int r) {
        rollNo = r;     }
    void showStudent() {
        cout << "Roll No: " << rollNo << endl;     } };
```

```
class Marks {
protected:
    int m1, m2;
public:
    void setMarks(int x, int y) {
        m1 = x; m2 = y;
    }
    void showMarks() {
        cout << "Marks: " << m1 << ", " << m2 << endl;
    }
};
// Hybrid Inheritance: Derived from Student and Marks
class Result : public Student, public Marks {
    int total;
public:
    void calculate() {
        total = m1 + m2;
    }
    void showResult() {
        showPerson();
        showStudent();
        showMarks();
        cout << "Total = " << total << endl;
} };
```

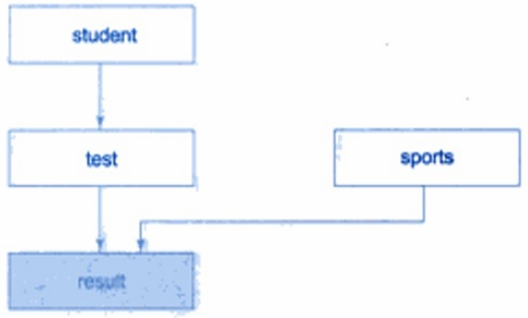


Fig. 8.11 ⇔ *Multilevel, multiple inheritance*

```
int main() {
    Result r;
    r.setPerson("Alice", 20);
    r.setStudent(101);
    r.setMarks(80, 90);
    r.calculate();
    r.showResult();
    return 0;}
```

Virtual Base Classes

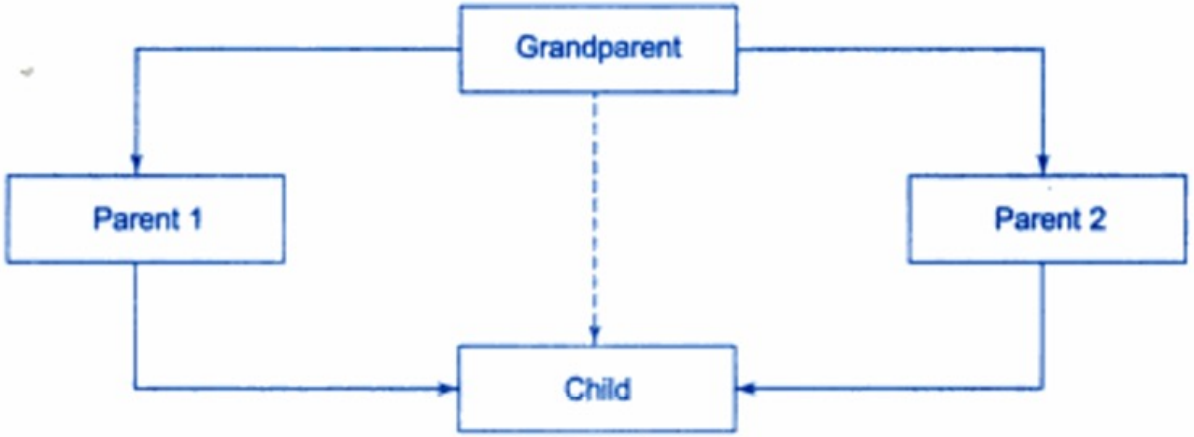


Fig. 8.12 ⇔ *Multipath inheritance*

Problems with Multiple Inheritance

- **Ambiguity Problem**

If two base classes have a function with the same name, the derived class cannot decide which one to call.

- **Diamond Problem (Duplicate Inheritance of Same Base)**

When two classes inherit from the same base class, and then a third class inherits from both of them, the base class members are inherited **twice**.

Solution → Virtual Base Classes

To solve the **diamond problem**, we declare the base class as **virtual**:

When a class is made a **virtual** base class, C++ takes necessary care to see that only one copy of that class is inherited, regardless of how many inheritance paths exist between the virtual base class and a derived class.

Abstract Classes

- An **abstract class** is a class that cannot be instantiated (you cannot create objects of it).
- It is designed only to act as a **base class** (to be inherited by other classes).
- An **abstract class** is a class that has at least **one pure virtual function**.

Constructors in Derived Classes

- **Order of Execution:**
  - When an object of a derived class is created, the base class's constructor is executed first, followed by the derived class's constructor.
  - When an object of a derived class is destroyed, its destructor is called first, then that of the base class.
- **Default Constructor Invocation:**
  - If a base class has a default (no-argument) constructor, and the derived class does not explicitly call a specific base class constructor, the base class's default constructor is automatically invoked.
- **Explicit Base Class Constructor Call:**
  - If a base class has a **parameterized constructor**, the derived class must explicitly call it using an **initializer list**.

Cont..

The general form of defining a derived constructor is:

```
Derived-constructor        (Arglist1, Arglist2, ...  ArglistN,   Arglist(D)

        base1(arglist1),
        base2(arglist2),
        .....
        .....
        .....
        baseN(arglistN),          arguments for base(N)
        {
                Body of derived constructor
        }
```

Example:

```
D(int a1, int a2, float b1, float b2, int d1):
A(a1, a2),          /* call to constructor A */
B(b1, b2)          /* call to constructor B */
{
      d = d1;      // executes its own body
}
```

```
.....
D objD(5, 12, 2.5, 7.54, 30);
.....
```

5	⟶	a1
12	⟶	a2
2.5	⟶	b1
7.54	⟶	b2
30	⟶	d1

Table 8.2 *Execution of base class constructors*

Method of inheritance	***Order of execution***

Class B: public A
{
};
	A( ) ; base constructor
	B( ) ; derived constructor

class A : public B, public C
{
};
	B( ) ; base(first)
	C( ) ; base(second)
	A( ) ; derived

class A : public B, virtual public C
{
};
	C( ) ; virtual base
	B( ) ; ordinary base
	A( ) ; derived

Constructors in Derived Class Example

```
class alpha
{
    int x;
  public:
    alpha(int i)
    {
        x = i;
        cout << "alpha initialized \n";
    }
    void show_x(void)
    { cout << "x = " << x << "\n"; }
};

class beta
{
    float y;
  public:
    beta(float j)
    {
        y = j;
        cout << "beta initialized \n";
    }
    void show_y(void)
    { cout << "y = " << y << "\n"; }
};
```

```
class gamma: public beta, public alpha
{
    int m, n;
  public:
    gamma(int a, float b, int c, int d):
        alpha(a), beta(b)
    {
        m = c;
        n = d;
        cout << "gamma initialized \n";
    }

    void show_mn(void)
    {
        cout << "m = " << m << "\n"
             << "n = " << n << "\n";
    }
};

int main()
{
    gamma g(5, 10.75, 20, 30);
    cout << "\n";
    g.show_x();
    g.show_y();
    g.show_mn();

    return 0;
}
```

```
beta initialized
alpha initialized
gamma initialized

x = 5
y = 10.75
m = 20
n = 30
```

Cont..

- A **constructor initializer list** is a way to **initialize data members** of a class **before the constructor body executes**.

```
ClassName(parameters) : member1(value1), member2(value2),
... {
    // constructor body
}
```

```
#include <iostream>
using namespace std;

class Student {
    int roll;
    string name;
public:
    Student(int r, string n) : roll(r), name(n) {
        cout << "Student created with roll = " << roll << ", name = " << name << endl;
    } };
int main() {
    Student s1(101, "Alice");
    return 0;    }
```

Member classes: nesting of classes

```
class alpha {....};
class beta {....};
class gamma
{
        alpha a;
        beta b;
        .....
};
```

All objects of the **gamma** class will contain the objects **a** and **b**. This kind of relationship is called **containership** or **nesting**.

A nested object is created in two stages. First, the member objects are created using their respective constructors and then the other 'ordinary' members are created.
This means, constructors of all the member objects should be called before its own constructor body is executed.
This is accomplished using an initialization list in the constructor of the nested class.

```
class gamma
{
    .....
    alpha a;        // a is object of alpha
    beta b;         // b is object of beta
  public:
    gamma(arglist): a(arglist1), b(arglist2)
    {
            // constructor body
    }
};
```

```
gamma(int x, int y, float z) : a(x), b(x,z)
{
        Assignment section(for ordinary other members)
}
```